Федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Лабораторная работа #5

Вариант #258

По дисциплине

Программирование

Выполнил студент группы R3138

Глазков В.Н.

Преподаватель:

Кобелев Р.П.

г. Санкт-Петербург

2025 г.

Текст задания

2-й семестр (весна)

Лабораторная работа #5

Введите вариант: 258

Внимание! У разных вариантов разный текст задания!

Реализовать консольное приложение, которое реализует управление коллекцией объектов в интерактивном режиме. В коллекции необходимо хранить объекты класса `Product`, описание которого приведено ниже.

Разработанная программа должна удовлетворять следующим требованиям:

- Класс, коллекцией экземпляров которого управляет программа, должен реализовывать сортировку по умолчанию.
- Все требования к полям класса (указанные в виде комментариев) должны быть выполнены.
- Для хранения необходимо использовать коллекцию типа `java.util.PriorityQueue`
- При запуске приложения коллекция должна автоматически заполняться значениями из файла.
- Имя файла должно передаваться программе с помощью: **аргумент командной строки**.
- Данные должны храниться в файле в формате `json`
- Чтение данных из файла необходимо реализовать с помощью класса `java.io.BufferedInputStream`
- Запись данных в файл необходимо реализовать с помощью класса `java.io.BufferedWriter`
- Все классы в программе должны быть задокументированы в формате javadoc.
- Программа должна корректно работать с неправильными данными (ошибки пользовательского ввода, отсутсвие прав доступа к файлу и т.п.).

В интерактивном режиме программа должна поддерживать выполнение следующих команд:

- `help` : вывести справку по доступным командам
- `info` : вывести в стандартный поток вывода информацию о коллекции (тип, дата инициализации, количество элементов и т.д.)
- `show` : вывести в стандартный поток вывода все элементы коллекции в строковом представлении
- `add {element}` : добавить новый элемент в коллекцию
- `update id {element}` : обновить значение элемента коллекции, id которого равен заданному
- `remove_by_id id` : удалить элемент из коллекции по его id
- `clear` : очистить коллекцию
- `save` : сохранить коллекцию в файл
- `execute_script file_name` : считать и исполнить скрипт из указанного файла. В скрипте содержатся команды в таком же виде, в котором их вводит пользователь в интерактивном режиме.
- `exit` : завершить программу (без сохранения в файл)
- `remove_first` : удалить первый элемент из коллекции
- `add_if_min {element}` : добавить новый элемент в коллекцию, если его значение меньше, чем у наименьшего элемента этой коллекции
- `history` : вывести последние 13 команд (без их аргументов)
- `sum_of_price` : вывести сумму значений поля price для всех элементов коллекции
- `average_of_price` : вывести среднее значение поля price для всех элементов коллекции
- `filter_by_unit_of_measure unitOfMeasure` : вывести элементы, значение поля unitOfMeasure которых равно заданному

Формат ввода команд:

- Все аргументы команды, являющиеся стандартными типами данных (примитивные типы, классы-оболочки, String, классы для хранения дат), должны вводиться в той же строке, что и имя команды.
- Все составные типы данных (объекты классов, хранящиеся в коллекции) должны вводиться по одному полю в строку.
- При вводе составных типов данных пользователю должно показываться приглашение к вводу, содержащее имя поля (например, "Введите дату рождения:")
- Если поле является enum'ом, то вводится имя одной из его констант (при этом список констант должен быть предварительно выведен).
- При некорректном пользовательском вводе (введена строка, не являющаяся именем константы в enum'е; введена строка вместо числа; введённое число не входит в указанные границы и т.п.) должно быть показано сообщение об ошибке и предложено повторить ввод поля.
- Для ввода значений null использовать пустую строку.
- Поля с комментарием "Значение этого поля должно генерироваться автоматически" не должны вводиться пользователем вручную при добавлении.

Описание хранимых в коллекции классов:

```
public class Product {
    private Integer id; //Поле не может быть null, Значение поля должно быть больше 0, Значение этого поля должно быть уникальным, Значение этого поля должно ген
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.time.ZonedDateTime creationDate; //Поле не может быть null, Значение этого поля должно генерироваться автоматически
    private Float price; //Поле может быть null, Значение поля должно быть больше 0
    private String partNumber; //Значение этого поля должно быть уникальным, Длина строки не должна быть больше 83, Длина строки должна быть не меньше 22, Поле н
    private UnitOfMeasure unitOfMeasure; //Поле не может быть null
    private Person owner; //Поле не может быть null
}
public class Coordinates {
    private Integer x; //Поле не может быть null
    private Float y; //Поле не может быть null
}
public class Person {
    private String name; //Поле не может быть null, Строка не может быть пустой
    private java.util.Date birthday; //Поле может быть null
    private String passportID; //Поле может быть null
    private Location location; //Поле не может быть null
}
public class Location {
    private float x;
    private Integer y; //Поле не может быть null
    private long z;
    private String name; //Длина строки не должна быть больше 986, Поле не может быть null
}
public enum UnitOfMeasure {
    KILOGRAMS,
    LITERS,
    MILLILITERS,
    MILLIGRAMS;
}
```

Отчёт по работе должен содержать:

1. Текст задания.
2. Диаграмма классов разработанной программы.
3. Исходный код программы.
4. Выводы по работе.

Вопросы к защите лабораторной работы:

1. Коллекции. Сортировка элементов коллекции. Интерфейсы `java.util.Comparable` и `java.util.Comparator`.
2. Категории коллекций - списки, множества, очереди, отображения (словари). Интерфейсы `java.util.List`, `java.util.Set`, `java.util.Queue`, `java.util.Map` и их реализации.
3. Параметризованные типы. Создание параметризуемых классов. Wildcard-параметры.
4. Классы-оболочки. Назначение, область применения, преимущества и недостатки. Автоупаковка и автораспаковка.
5. Потоки ввода-вывода в Java. Байтовые и символьные потоки. "Цепочки" потоков (Stream Chains).
6. Работа с файлами в Java. Класс `java.io.File`.
7. Пакет `java.nio` - назначение, основные классы и интерфейсы.
8. Утилита `javadoc`. Особенности автоматического документирования кода в Java.

Диаграмма классов реализованной объектной модели

https://github.com/RYAZHENKA-11/itmo-java-labs/blob/main/lab5-258/uml.png

Исходный код программы

https://github.com/RYAZHENKA-11/itmo-java-labs/tree/main/lab5-258

Вывод

В процессе выполнения лабораторной работы:

- Изучены принципы работы с коллекциями типа PriorityQueue и механизмы сортировки элементов по умолчанию с использованием интерфейса Comparable.
- Создана объектная модель данных с использованием record-типов (Product, Person, Coordinates, Location) для хранения информации о товарах и их владельцах.
- Реализован собственный JSON-парсер для чтения данных из файла через BufferedInputStream и записи через BufferedWriter в соответствии с требованиями задания.
- Разработана система команд интерактивного режима с использованием паттернов Command, Invoker и Factory для обработки пользовательского ввода.
- Использованы перечисления (enum UnitOfMeasure) для типизации единиц измерения товаров.

- Организована проверка всех полей на соответствие требованиям задания (уникальность id и partNumber, валидация диапазонов значений, обработка null-полей).
- Реализована устойчивость программы к неправильным данным и некорректному пользовательскому вводу с выводом информативных сообщений об ошибках.
- Все классы задокументированы в формате Javadoc.

В результате была разработана работоспособная программа для управления коллекцией товаров в интерактивном режиме, соответствующая всем требованиям задания. Все цели лабораторной работы достигнуты.